

Architecture & Design Document: RWA T-Bill Tokenization Platform

Course: Blockchain Technologies 2, Final Project

Scenario: Option C - RWA Tokenization Platform

Protocol: Tokenized short-duration U.S. T-Bill reserve model

Target network: Base Sepolia, with Anvil for local demos

Tooling: Foundry, Anvil, OpenZeppelin 5.6.1, Vite React, Wagmi, Viem, RainbowKit, The Graph

Team ownership: Tokhtamishov Bekhruz - contracts, security, tests, deploy. Zhangir Muslimov - frontend, subgraph, docs, presentation, gas report.

1. Executive Architecture Summary

The system implements a full-stack RWA protocol around a simplified tokenized T-Bill product. Authorized issuers mint `TBILL` only after off-chain reserve evidence exists. Investors can hold `TBILL`, deposit it into an ERC4626 vault, provide liquidity against a settlement token in a from-scratch AMM, and govern privileged protocol changes through OpenZeppelin Governor plus a two-day Timelock.

10

Production contracts across tokenization, DeFi, oracle, governance, treasury, and deployment infrastructure.

88

Foundry tests covering unit, fuzz, invariant, fork hooks, and vulnerability case studies.

92.31%

Line coverage for production `src/` contracts with mocks, scripts, and tests excluded.

Mandatory Requirement Coverage

Requirement area	Implementation
Advanced Solidity	UUPS <code>TBillToken</code> , V1 to V2 upgrade path, <code>ProtocolFactory</code> with CREATE and CREATE2, inline Yul math benchmarked against Solidity.
Token standards	<code>RwaGovernanceToken</code> as ERC20Votes + ERC20Permit, <code>ReserveCertificateNFT</code> as ERC721, <code>TBillVault</code> as ERC4626.
DeFi primitive	<code>RwaStableAMM</code> , built from scratch as constant product $x*y=k$ with 0.3% fee, slippage checks, deadlines, and LP accounting.
Oracles	<code>OracleAdapter</code> reads Chainlink-compatible price and reserve feeds with stale, zero, and negative answer reverts.
Indexing	The Graph subgraph indexes assets, transfers, vault actions, swaps, liquidity, proposals, votes, and treasury payments.
Governance	OpenZeppelin Governor, timestamp clock, 1 day voting delay, 1 week voting period, 4% quorum, 1% proposal threshold, and 2 day Timelock.
Layer 2	Deployment scripts target Base Sepolia and write deterministic deployment manifests. Anvil is used for reproducible local verification.
Frontend	Vite React dApp supports wallet connection, network detection, protocol reads, state-changing writes, governance voting, and subgraph activity sections.
DevOps	CI runs compile, test, coverage, formatting, Solhint, Slither, frontend build, and a no-Hardhat guard.

2. System Context Diagram

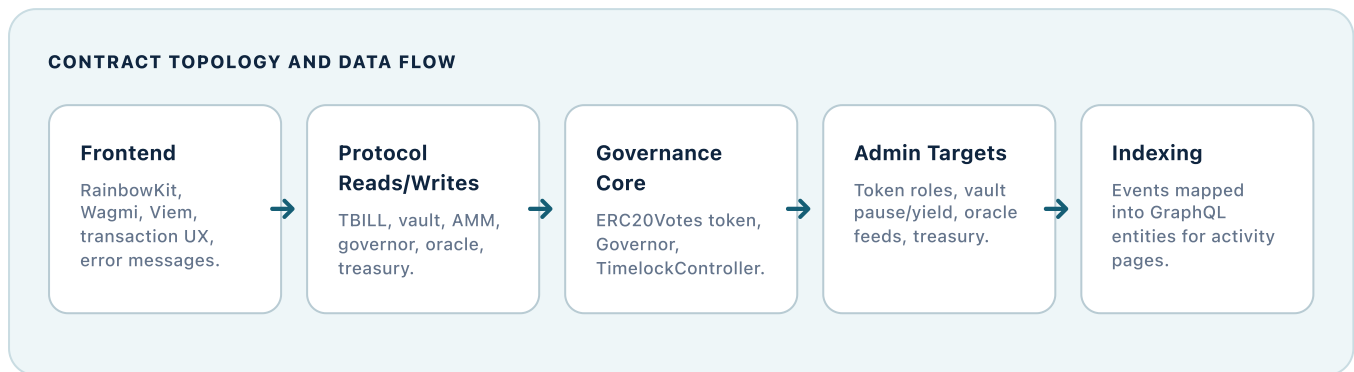


The protocol boundary is the deployed contract set plus the subgraph schema. The legal custody of the real-world T-Bill reserve remains an off-chain assumption. On testnet, the reserve feed is intentionally mock-backed because no official Base Sepolia T-Bill proof-of-reserve feed is assumed.

External Actors and Responsibilities

Actor / system	Responsibility	Trust level
Investor	Uses wallet, deposits, swaps, delegates, votes, and monitors proposals.	Does not need privileged roles.
Authorized issuer	Mints TBILL only after reserve evidence exists.	Trusted operational actor, removable by governance.
Governor voters	Approve or reject parameter changes, role changes, treasury actions, and upgrades.	Token-weighted governance with checkpointed voting power.
Timelock	Executes successful governance actions after a two-day delay.	Root admin after deployment.
Chainlink-compatible feeds	Provide latest price and reserve answers.	Trusted for correctness and freshness within configured max staleness.
The Graph	Indexes events for the frontend and demo queries.	Read-only convenience layer; contract state is source of truth.

3. Container and Component Architecture



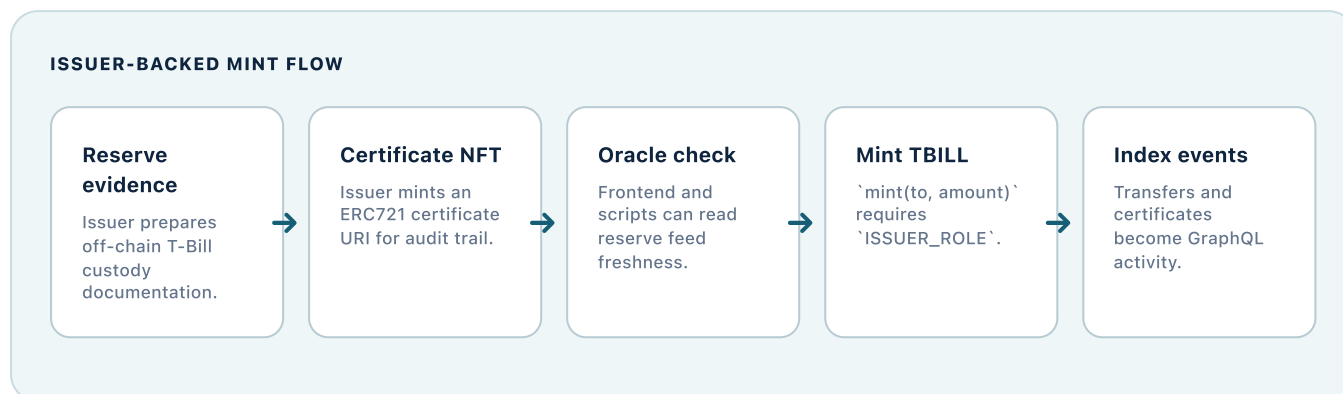
Contract Map

Contract	Pattern / standard	Primary responsibility
<code>TBillToken</code>	ERC20, UUPS, AccessControl	Asset-backed token minted by authorized issuers. Supports pause, burn, registry updates, and Timelock-controlled upgrade.
<code>TBillTokenV2</code>	UUPS upgrade target	Appends <code>complianceUri</code> after V1 storage and proves upgrade path without storage collision.
<code>RwaGovernanceToken</code>	ERC20Votes, ERC20Permit	Governance voting power, delegation, permit-based approvals, timestamp-based ERC6372 clock.
<code>ReserveCertificateNFT</code>	ERC721, AccessControl, Pausable	Issues reserve evidence certificates with metadata URIs for off-chain collateral documentation.
<code>TBillVault</code>	ERC4626, ReentrancyGuard	Tokenized vault for TBILL deposits and role-gated yield reporting.
<code>RwaStableAMM</code>	Custom AMM, ERC20 LP	Constant-product liquidity pool with 0.3% fee, slippage protection, deadlines, and LP share accounting.
<code>OracleAdapter</code>	Adapter / interface abstraction	Wraps Chainlink-compatible price and reserve feeds, enforcing max staleness and answer validity.
<code>ProtocolTreasury</code>	Pull payments	Holds ETH/ERC20 treasury assets and lets payees withdraw scheduled payments.
<code>ProtocolFactory</code>	Factory, CREATE, CREATE2	Deploys new UUPS asset proxies through both normal and deterministic deployment paths.
<code>RwaGovernor</code>	Governor + Timelock	Proposal lifecycle: propose, vote, queue, execute.

Proxy Layout and Admin Boundary

Only `TBillToken` is upgradeable. The proxy address is the user-facing asset contract. The implementation contract disables initializers in its constructor, and the proxy is initialized exactly once through `ERC1967Proxy` constructor calldata. The `UPGRADER_ROLE` is handed to the Timelock, so upgrades require a successful governance proposal and a two-day delay.

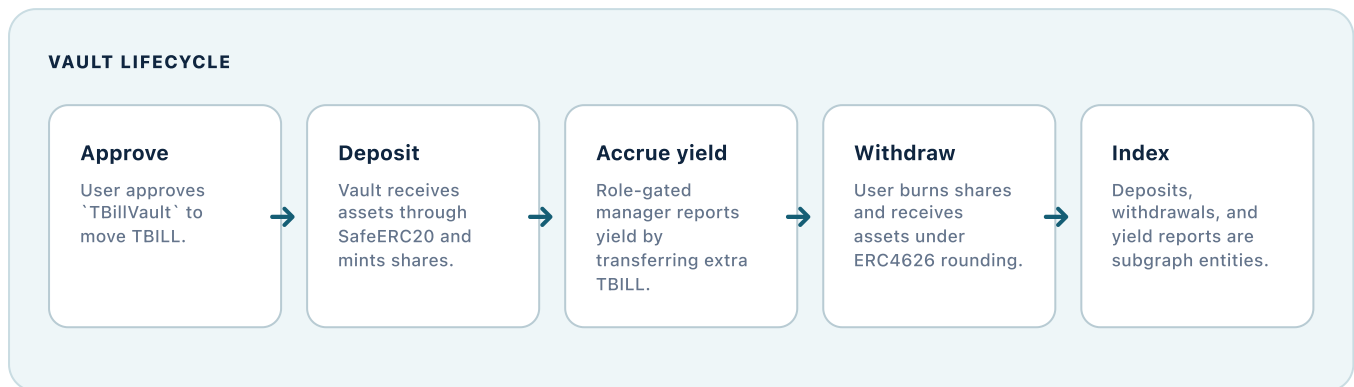
4. Critical User Flow 1 - Issuer Mint



Security properties:

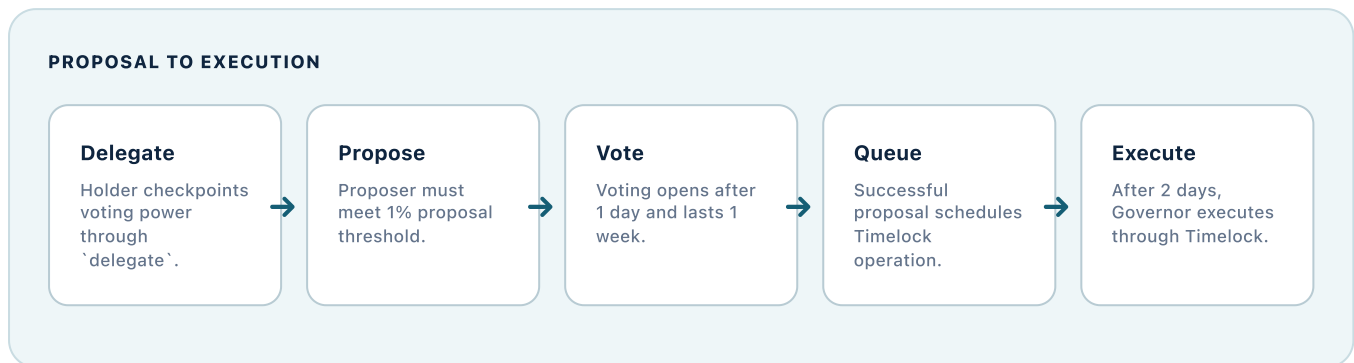
- Minting is role-gated through OpenZeppelin `AccessControl`.
- Governance can remove compromised issuers without upgrading the token.
- The ERC20 transfer event is indexed by the subgraph for off-chain reconciliation.
- The production design expects real reserve feeds before mainnet; the testnet path documents mock feeds explicitly.

5. Critical User Flow 2 - ERC4626 Deposit and Yield



The vault uses OpenZeppelin ERC4626 conversion logic and the tests include rounding-focused fuzz cases. All externally callable value-moving vault functions are `nonReentrant`, use SafeERC20, and can be paused for incident response.

6. Critical User Flow 3 - Governance Lifecycle



Governance controls the treasury and privileged protocol roles. The deployer is intended to be temporary and loses long-term authority after deployment handoff. `VerifyDeployment.s.sol` checks that the deployed system has the expected Timelock delay, Governor parameters, and role ownership.

7. Data Model and Storage Layout

Upgradeable Token Storage

Storage group	Field / owner	Collision risk analysis
OpenZeppelin	ERC20 balances, allowances, total supply, name, symbol	Managed by OpenZeppelin upgradeable contracts and inherited in stable order.
OpenZeppelin	Pausable state	Inherited before local fields and unchanged between V1 and V2.
OpenZeppelin	AccessControl role admin and membership maps	Inherited before local fields and unchanged between V1 and V2.
OpenZeppelin	UUPS and Initializable state	Required for proxy safety and upgrade authorization.
V1 local	<code>address registry</code>	First local project-defined variable.
V1 reserved	<code>uint256[49] __gap</code>	Reserved expansion space for future versions.
V2 local	<code>string complianceUri</code>	Appended after V1 fields; tests confirm balances, roles, and registry survive the upgrade.

Non-Upgradeable Contract Storage

Contract	Key state	Notes
<code>RwaGovernanceToken</code>	Balances, votes checkpoints, permit nonces, delegation state, roles.	Timestamp clock gives human-readable governance windows.
<code>ReserveCertificateNFT</code>	ERC721 ownership, token URIs, <code>nextTokenId</code> , roles, pause state.	Certificates are not a legal title transfer; they are reserve evidence references.
<code>TBillVault</code>	ERC4626 share balances, total supply, immutable TBILL asset, roles, pause/reentrancy flags.	ERC4626 accounting is tested under deposits, mints, withdrawals, redeems, and yield.
<code>RwaStableAMM</code>	Immutable tokens, reserves, last update timestamp, LP ERC20 balances and allowances.	<code>k</code> invariant is tracked in invariant tests after swaps.
<code>OracleAdapter</code>	Price feed, reserve feed, max staleness, roles.	Business logic reads adapter functions, not raw aggregator contracts.
<code>ProtocolTreasury</code>	ERC20 owed balances, ETH owed balances, roles.	Pull-over-push model avoids failed receiver loops.
<code>ProtocolFactory</code>	TBILL implementation address, role configuration.	Deterministic address prediction is exposed for CREATE2 deployments.
<code>RwaGovernor</code>	Proposal state, voting snapshots, quorum, Timelock operation IDs.	OpenZeppelin modules compose the full propose to execute lifecycle.

8. Roles, Permissions, and Trust Assumptions

Role / power	Holder after deployment	Capability	Risk if compromised	Mitigation
Timelock admin	Timelock	Root executor for privileged calls.	Malicious successful proposal can change parameters or move treasury.	2 day delay, public proposal trail, token voting checkpoints.
Token upgrader	Timelock	Authorizes <code>TBillToken</code> UUPS upgrades.	Bad implementation can break token behavior.	Upgrade proposal delay and V1 to V2 storage tests.
Issuer role	Authorized issuer	Mint <code>TBILL</code> against reserve evidence.	Over-minting if issuer is malicious or operationally compromised.	Governance can revoke role; reserve certificates and oracle data are auditable.
Vault manager	Timelock / configured manager	Report yield into ERC4626 vault.	False or unsupported yield reporting.	SafeERC20 transfer requires actual <code>TBILL</code> movement into vault.
Oracle admin	Timelock	Update feeds and max staleness.	Bad feed can distort protocol readouts.	Timelock delay and explicit stale/invalid answer reverts.
Treasury scheduler	Timelock	Schedule token or ETH payments.	Misallocation of treasury assets.	Pull payments and governance execution trail.
Pauser	Timelock	Pause token, certificate, and vault flows during incident response.	Governance can halt user flows.	Documented circuit breaker, delay for planned actions, limited to pausable contracts.

Trust assumptions:

- Off-chain custody, legal ownership, and reserve attestations are outside the smart-contract scope.
- The testnet deployment uses mock feeds for demo; mainnet deployment must replace them with official production feeds.
- Base Sepolia gas and final addresses depend on deploy secrets that are intentionally not committed.
- A compromised deployer before role handoff is a serious risk; the deployment script and verification script are designed to make handoff explicit.

9. Design Patterns and Architecture Decisions

Pattern	Where used	Why it is needed
Factory pattern	<code>ProtocolFactory</code>	Required to deploy asset proxies through CREATE and CREATE2, including deterministic prediction.
Proxy / UUPS	<code>TBillToken</code>	RWA compliance metadata can evolve while keeping a stable token address.
Checks-Effects-Interactions	Treasury withdrawals, AMM flows	State is updated before external value transfer or protected by reentrancy guard.
Pull-over-push payments	<code>ProtocolTreasury</code>	Payees withdraw, so a receiver cannot block a payout batch.
Role-based access control	Token, vault, oracle, treasury	All privileged functions have explicit OpenZeppelin role checks.
Pausable / circuit breaker	Token, NFT, vault	Governance can stop flows during compliance or oracle incidents.
Oracle adapter abstraction	<code>OracleAdapter</code>	Feed contracts can be replaced without rewriting business logic.
Timelock	Governance admin layer	Adds a 2 day public delay before privileged actions.
ReentrancyGuard	Vault, AMM, fixtures	Protects value-moving entry points and demonstrates the fixed vulnerability case study.

ADR-001: UUPS Only for the RWA Asset Token

Context: RWA compliance metadata can change, but upgradeability increases audit risk. Options considered were immutable token, transparent proxy, or UUPS proxy. Decision: use UUPS only for `TBillToken`; keep the vault, AMM, treasury, oracle, and Governor non-upgradeable. Consequence: the upgrade surface is isolated and governance-controlled.

ADR-002: Timelock as Long-Term Administrator

Context: direct owner or deployer admin powers would violate the governance requirement and create an instant-change risk. Decision: privileged roles are handed to `TimelockController`. Consequence: users and reviewers can monitor queued operations before they execute.

ADR-003: ERC4626 for Yield Representation

Context: tokenized yield needs consistent share accounting. Options were custom shares or ERC4626. Decision: ERC4626. Consequence: auditors can rely on a known standard, and the test suite can focus on rounding, share conversion, and withdrawal behavior.

ADR-004: From-Scratch Constant Product AMM

Context: the course requires a DeFi primitive built from scratch. Decision: implement $x*y=k$ with 0.3% fee, LP shares, deadline checks, and slippage limits. Consequence: the surface is smaller than a full Uniswap fork but still demonstrates reserves, liquidity shares, swaps, and invariants.

ADR-005: Chainlink-Compatible Adapter

Context: production feeds and testnet mocks should share the same interface. Decision: write an adapter around `latestRoundData` with staleness, zero answer, and negative answer checks. Consequence: tests can use mocks while production can replace feed

addresses through governance.

ADR-006: Pull Payments for Treasury

Context: pushing payments to arbitrary receivers can fail or re-enter. Decision: treasury records owed balances and lets receivers withdraw. Consequence: the treasury has clearer accounting and a narrower external-call surface.

10. Deployment and Verification Model

The deployment system is Foundry-only. `Deploy.s.sol` deploys mocks when needed, deploys the full protocol stack, performs role handoff, delegates initial governance voting power, and writes `deployments/<chainId>.json`. `VerifyDeployment.s.sol` reads that manifest and verifies the critical post-deployment invariants.

Script	Purpose
<code>script/Deploy.s.sol</code>	Idempotent Anvil/Base Sepolia deployment and manifest generation.
<code>script/UpgradeTBillToken.s.sol</code>	Governance-compatible upgrade path from V1 to V2.
<code>script/VerifyDeployment.s.sol</code>	Confirms Timelock delay, Governor parameters, role handoff, and deployed addresses.
<code>script/GasCompare.s.sol</code>	Benchmarks gas-sensitive protocol operations and writes the gas report.

Required real deployment environment variables are documented in `.env.example`: `BASE_SEPOLIA_RPC_URL`, `PRIVATE_KEY`, `BASESCAN_API_KEY`, `WALLETCONNECT_PROJECT_ID`, `SUBGRAPH_DEPLOY_KEY`, and `MAINNET_RPC_URL`.

11. Frontend and Subgraph Architecture

The frontend is intentionally an operational protocol dashboard, not a marketing page. It exposes wallet connection, network switching, token balances, voting power, delegate address, vault shares/assets, AMM reserves, oracle data, issuer actions, proposal activity, and user-triggered write flows.

The subgraph indexes events into at least the required four entities and provides documented GraphQL queries. Indexed sections are used for activity and proposal state, while critical balances and protocol state are read directly from contracts.

Frontend capability	Source of truth	Notes
Wallet and network status	Wagmi / wallet provider	Wrong network prompts the user to switch.
Token balances	Contracts through Viem	Reads TBILL, voting power, delegate, vault shares, and AMM reserves.
State-changing actions	Contracts through wallet	Deposit, withdraw, liquidity, swap, delegate, vote, and issuer mint flows.
Activity feed	The Graph	Uses indexed events rather than direct contract reads.
Error messages	Frontend transaction wrapper	Converts raw RPC and wallet rejection errors into readable user messages.

12. Engineering Readiness Checklist

Area	Status
Compile	<code>forge build</code> passes.
Tests	<code>forge test</code> passes with 88 tests.
Coverage	<code>forge coverage --ir-minimum</code> reports 92.31% line coverage for production contracts.
Formatting	<code>forge fmt --check</code> and Prettier are included in local checks and CI.
Security tooling	Slither is configured in CI and expected to report zero High and zero Medium findings.
No Hardhat	Repository guard fails if Hardhat config, dependency, script, or CI usage is introduced.
Deployment	Anvil deployment is verified; Base Sepolia deployment requires local secrets.
Documentation	Architecture, audit, gas, coverage, deployment verification, and presentation PDFs exist.